

RIDESHARE PLATFORM

Capstone Project Documentation & Technical Guide

System Architecture & Backend Logic

A comprehensive overview of a complete, dual-facing ridesharing solution (similar to Uber or Lyft). This system features a real-time NodeJS backend, interactive maps for ride matching and tracking, and dedicated front-end applications for both riders and drivers.

This document is written in clear, simple language to explain the code structure, used packages, database schemas, and the real-time workflows that connect users.

Date: July 3, 2026

Prepared for: Capstone Project Submission
Generated by: Antigravity AI Coding Assistant

1. Project Architecture & Overview

This project is an end-to-end RideSharing Application (an Uber/Lyft clone) built to demonstrate modern full-stack web development. The project is split into three main parts: a centralized Backend API Server and two distinct Frontend Web Applications for Riders and Drivers. This split allows both user roles to have tailored user interfaces and clean, non-interfering workflows.

The Three Main Applications

- * **1. Backend Server:** A NodeJS & Express server that manages database operations, secures accounts, performs ride calculations, and handles real-time synchronization between clients using WebSockets.
- * **2. Rider Client App:** A React-based web dashboard where riders log in, view live maps, enter pickup and dropoff destinations, request rides, review the calculated fare and distance, track their driver's car marker, and rate completed trips.
- * **3. Driver Client App:** A React-based web dashboard where drivers toggling availability see a stream of nearby ride requests, accept them, update the progress of active rides (arrived, in-progress, completed), view location markers, and monitor their dashboard stats (total rides, rating, and earnings).

Key Features of the System

- * **Real-Time Tracking:** Sockets broadcast GPS coordinates instantly between driver and rider maps.
- * **Dynamic Fare Calculation:** Automated distance calculation and fare estimation based on destination coordinates.
- * **Secure Auth:** Password encryption on signup, with role-based JWT validation for drivers and riders.
- * **Interactive Mapping:** Uses OpenStreetMap layers with Leaflet to select and track locations visually.

2. Package Manifest (Packages Used)

Each application in the workspace contains its own package.json. The packages listed below were chosen to satisfy security, validation, real-time communications, database mapping, and mapping visualization needs.

A. Backend Server Packages

- * **express:** The minimalist web framework used to configure and run the REST API endpoints.
- * **mongoose:** An Object Data Modeling (ODM) library for MongoDB. It enforces strict schemas and simplifies database queries.
- * **socket.io:** Enables real-time, bi-directional communication. Used to send ride alerts and live locations.
- * **bcryptjs:** A cryptographic package used to hash and salt passwords, protecting user data in database storage.
- * **jsonwebtoken (jwt):** Signs and verifies JSON Web Tokens to establish secure, stateless sessions for users.
- * **cors:** Enables Cross-Origin Resource Sharing. Allows frontend apps running on ports 5173/5174 to call the API server.
- * **dotenv:** Loads environment configuration variables from a local `.env` file (e.g. database connection string, port, secrets).
- * **express-validator:** A validation middleware. It blocks invalid payloads (like short passwords or malformed email strings) before they hit database queries.
- * **nodemon (dev):** Restarts the server processes automatically on file updates, speeding up developer productivity.

B. Frontend Applications Packages (Rider & Driver Apps)

- * **react & react-dom:** The building block libraries for constructing reactive, modular user interfaces.
- * **react-router-dom:** Handles URL navigation and private route protection (blocking unauthorized users from home screens).
- * **axios:** A clean, promise-based HTTP library used to submit login forms, fetch ride history, and trigger api updates.
- * **leaflet:** The industry-standard open-source map engine that does the heavy-lifting of rendering geographical maps.
- * **react-leaflet:** A set of wrapper bindings that allow Leaflet maps to be integrated as React components (e.g. `<MapContainer>`, `<Marker>`).
- * **socket.io-client:** Connects the client app to the server sockets. Listens for live coordinates and statuses.
- * **vite (dev):** The lightning-fast build bundler that runs the frontend apps locally during development.

3. Database Structure & Schemas

The project uses MongoDB as its database, which is a document-oriented database. Through Mongoose, we enforce schemas on the tables (collections) to structure relations. Below are the structures for the three core schemas in the system.

[Schema Snippet] User Model - Defines account credentials and identity.

```
UserSchema = {
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }, // stored as bcrypt hash
  role:     { type: String, enum: ['rider', 'driver'], default: 'rider' }
}
```

[Schema Snippet] Driver Model - Extends User info with vehicle state and earnings.

```
DriverSchema = {
  user:      { type: ObjectId, ref: 'User', unique: true },
  vehicleInfo: { make, model, plate, color },
  location:   { lat: Number, lng: Number }, // live GPS coordinate
  isAvailable: { type: Boolean, default: false },
  rating:     { type: Number, default: 5.0 },
  totalRides: { type: Number, default: 0 },
  totalEarnings: { type: Number, default: 0 }
}
```

[Schema Snippet] Ride Model - Tracks the journey, transactions, and state changes.

```
RideSchema = {
  rider:      { type: ObjectId, ref: 'User', required: true },
  driver:     { type: ObjectId, ref: 'Driver', default: null },
  pickup:     { address: String, lat: Number, lng: Number },
  dropoff:    { address: String, lat: Number, lng: Number },
  status:     { type: String, enum: ['pending', 'accepted', 'in_progress', 'completed',
'cancelled'] },
  fare:       Number,
  distance:   Number, // calculated in km
  riderRating: Number // feedback rating from 1 to 5
}
```

4. How the Backend Works (Logic & Flow)

Here is the step-by-step description of how the backend handles authentication, ride bookings, real-time GPS synchronization, and transactions in simple words.

A. Registration & Authentication Flow

When a user signs up, the backend accepts their username and password. It runs validation checks, hashes the password using the bcryptjs library (so password strings are never saved in plain text), and creates a User document. For drivers, it also creates an empty Driver document to track vehicle information.

When logging in, the server checks the username, compares the password hash, and signs a JSON Web Token (JWT). This token contains the user's ID, username, and role. The client stores this token in localStorage. For any protected action, the client sends this token in its headers. The backend auth middleware intercepts the request, decrypts the token, checks the role, and allows or blocks the request.

B. Ride Creation & Fare Calculation

When a Rider selects a pickup and dropoff location on the map, the Rider app sends the coordinate pairs to the backend. The backend uses the Haversine Formula (a mathematical equation that calculates the great-circle distance between two points on a sphere using latitude and longitude) to calculate the distance in kilometers. It then calculates the fare using a simple formula: a base price of \$2.50, plus \$1.20 per kilometer.

The server creates a new Ride record with status 'pending'. It then broadcasts a message via WebSockets ('ride:new_request') to all active drivers connected to the platform. This makes the request pop up instantly on the dashboards of nearby drivers.

5. Live Synchronization & Ride Completion

C. Acceptance & WebSocket Rooms

When a driver accepts a pending ride, the driver's client sends a PATCH request to the backend. The backend checks if the ride is still pending. If yes, it links the driver's ID to the ride and updates the status to 'accepted'.

To update clients without spamming everyone, the backend uses Socket.IO 'Rooms':

- Upon logging in, every user joins a room named '<role>:<userId>' (e.g. 'rider:60b784a9...').
- The server emits a 'ride:status_update' event targeting only the rider's personal room. The Rider app hears this update and immediately redraws the UI, showing driver name and car details.
- The server also emits a global 'ride:accepted' event to all other drivers so the ride disappears from their queue.

D. Live Tracking (GPS Updates)

Once a ride is active, the driver's app uses browser location APIs to fetch their GPS coordinates and sends them to the socket server via 'driver:location_update'.

The socket server receives the coordinates, saves them to the Driver's document, and broadcasts them. It emits 'ride:<rideId>:driver_location' which is listened to only by the specific rider tracking that ride. This allows the rider's map marker to smoothly glide along the map roads in real-time.

E. Completion, Earnings & Ratings

When the driver reaches the destination, they press 'Complete Trip' in their app. The backend:

1. Updates the Ride document status to 'completed' and sets the completion timestamp.
2. Adds the calculated fare to the driver's 'totalEarnings' database field and increments their 'totalRides' count.
3. Sends a socket update to the rider telling them the ride is completed.

Upon receiving this, the rider app shows a rating screen. When the rider submits a rating (1-5 stars), the backend re-calculates the driver's average rating using a weighted formula:

$$\text{New Rating} = ((\text{Old Average} * \text{Previous Ride Count}) + \text{New Rating Value}) / (\text{Ride Count} + 1).$$

This keeps ratings accurate and prevents recent ratings from distorting historical scores.

Summary: This architecture utilizes REST API endpoints for non-realtime state changes (logins, requests, completion) and WebSockets (Socket.IO) for high-frequency updates (GPS, ride requests, cancellations), establishing a highly performant, scalable, and responsive application.